



VISITOR

Padrões Comportamentais (GoF)

Lorenzo Fabro

O QUE É O VISITOR?

O Visitor é um padrão de projeto comportamental do GoF.

Ele serve para adicionar novas operações aos objetos sem precisar modificar suas classes.

Por exemplo, temos os produtos Livro e Eletrônico. Em vez de colocar o cálculo de imposto dentro de cada produto, criamos um Visitor responsável por calcular o imposto de cada um.

Assim, podemos criar outros Visitors para fazer coisas diferentes, como gerar relatórios ou calcular descontos, mantendo as classes dos produtos mais simples.

A solução do Visitor:

Colocar essas operações FORA das classes dos produtos — em classes "visitantes" separadas.

UMA COMPARAÇÃO BEM FÁCIL:

Livro

R\$ 100

Eletrônico

R\$ 1000

ImpostoVisitor

**Uma pessoa especializada que visita
cada produto e calcula o imposto certo.**

Imagine uma loja com dois produtos:

Livro R\$ 100

Eletrônico R\$ 1.000

Agora precisamos calcular o imposto de cada produto.

Em vez de cada produto calcular seu próprio imposto,
usamos um ImpostoVisitor.

Livro → 5% de imposto = R\$ 5

Eletrônico 15% de imposto = R\$ 150

O Visitor é como uma pessoa especializada que visita
cada produto e faz a operação necessária.

IDEIA PRINCIPAL:

A principal ideia do Visitor é separar as operações dos objetos.

Por exemplo:

Hoje: calcular imposto → ImpostoVisitor

Amanhã: gerar relatório → RelatorioVisitor

Depois: calcular desconto → DescontoVisitor

Assim, podemos adicionar novas operações sem precisar modificar as classes Livro e Eletronico.



ESTRUTURA DO VISITOR:

O padrão possui algumas partes principais.

A interface Visitor define as operações que podem ser realizadas em cada tipo de objeto.

O ImpostoVisitor implementa essas operações e define, por exemplo, quanto cada produto deve pagar de imposto.

A interface Produto possui o método accept(), que permite que o objeto receba um Visitor.

Por fim, Livro e Eletronico implementam esse método e informam ao Visitor qual tipo de objeto está sendo visitado.

Essa comunicação entre accept() e visit() é o que permite o funcionamento do Double Dispatch

«interface» Visitor

+ visit(Livro)
+ visit(Eletronico)

«interface» Produto

+ accept(Visitor)

ImpostoVisitor

implementa visit(Livro) e visit(Eletronico), cada um com sua regra de cálculo.

Livro / Eletrônico

implementam accept(v) chamando v.visit(this) — o "double dispatch".

COMO FUNCIONA O DOUBLE DISPATCH?

O Double Dispatch é uma parte importante do funcionamento do Visitor.

Primeiro, o Visitor é enviado para o produto:

```
livro.accept(visitor)
```

O Livro recebe o Visitor e chama:

```
visitor.visit(this)
```

Como o this representa um Livro, o programa sabe que deve executar:

```
visit(Livro)
```

Se o objeto fosse um Eletrônico, seria executado:

```
visit(Eletronico)
```

Assim, o mesmo Visitor consegue realizar operações diferentes de acordo com o tipo do objeto visitado.

EXEMPLO EM JAVA:

```
interface Visitor {  
  
    void visit(Livro livro);  
  
    void visit(Eletronico eletronico);  
  
}  
  
interface Produto {  
  
    void accept(Visitor visitor);  
}  
  
class Livro implements Produto {  
  
    public void accept(Visitor v) { v.visit(this); } // this é um Livro  
  
}  
  
class ImpostoVisitor implements Visitor {  
  
    public void visit(Livro l) { /* 5% de imposto */ }  
  
    public void visit(Eletronico e) { /* 15% de imposto */ }  
  
}
```

EXEMPLO EM JAVA:

Neste exemplo, usamos Java para mostrar como o Visitor funciona na prática.

A interface Visitor possui um método visit() para cada tipo de produto.

A interface Produto possui o método accept(), que permite receber um Visitor.

As classes Livro e Eletronico implementam accept() e passam elas mesmas para o Visitor.

Por último, o ImpostoVisitor contém as regras para calcular o imposto de cada produto.

Assim, cada classe fica responsável por sua função e o cálculo do imposto fica separado dos produtos.

RESULTADO, PRÓS/CONTRAS:

RESULTADO

Ao executar o programa, o mesmo Visitor pode visitar produtos diferentes e aplicar a regra correta.

Livro:

$\text{R\$ } 100 \times 5\% = \text{R\$ } 5$

Eletrônico:

$\text{R\$ } 1.000 \times 15\% = \text{R\$ } 150$

PRÓS

✓ Facilita adicionar novas operações, pois podemos criar novos Visitors.

✓ Evita modificar os produtos sempre que uma nova operação é criada.

✓ Mantém as classes dos produtos mais simples.

CONTRAS

✗ Se criarmos um novo tipo de produto, como Alimento, precisamos atualizar os Visitors para saber como trabalhar com esse novo tipo.



REFERÊNCIAS

GAMMA, Erich et al. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.

FREEMAN, Eric; ROBSON, Elisabeth. Head First Design Patterns. O'Reilly Media.

Refactoring.Guru — Visitor Design Pattern.